

Aprendizagem Aplicada à Segurança

Class 2: SPAM Detection I, from Naive Bayes to Logistic Regression

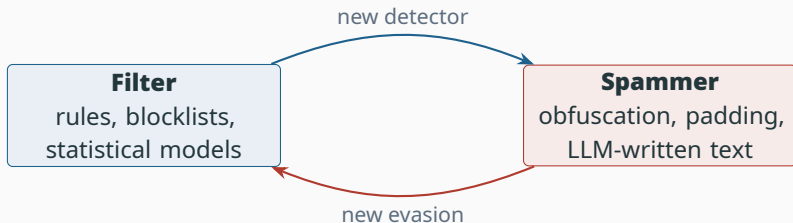
Mário Antunes

September 25, 2026

Universidade de Aveiro

Spam: an adversary, not a data set

- **Spam** is unsolicited bulk messaging; it carries advertising, fraud, phishing links and malware.
- A filter and a spammer are locked in an **arms race**: every new defence creates a new evasion.



- This class: **build** the detector, rung by rung. Next: **break** it.

From rules to statistics

- **Rules and blocklists** (1990s): keywords, sender lists. Precise, but brittle and easy to dodge (f r33).
- **Bayesian filtering** (2002): learn word statistics from *your own* mail (Graham 2002).
- Then linear models and ensembles; next class, **embeddings** from pre-trained language models.

Era	Representation	Typical model
Rules	keywords	if-then
2002-	word counts, TF-IDF	Naive Bayes, logistic regression
2016-	sub-word embeddings	fastText (Joulin et al. 2017)
2019-	sentence embeddings	small language models (Reimers and Gurevych 2019)

The task: binary text classification



- **Positive class = spam** (rare). Two errors, two costs:
 - **false positive:** a real message is blocked (users lose trust);
 - **false negative:** spam reaches the inbox (the filter fails).
- Most of this class is about **how to turn text into $\mathbf{x}$** , and which f_{θ} to use.

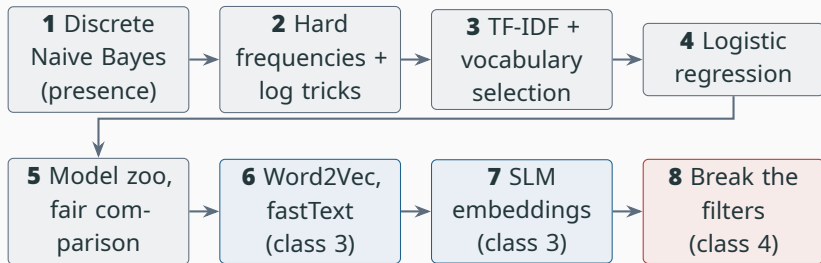
Data and evaluation protocol

SMS Spam Collection (Almeida et al. 2011): 5,574 English SMS, labelled ham or spam.

Step	What we do	Why
De-duplicate	5,574 $\rightarrow$ 5,119 unique	copies in train and test inflate the score
Stratified split	4,095 train, 1,024 test (129 spam)	keeps 12.6% spam in both
Fit on train only	vocabulary, IDF, embeddings	no information from the test set
Metrics	precision, recall, F1, MCC, PR-AUC	accuracy hides the class imbalance

- A filter that says “ham” to everything scores 87% accuracy and catches **no** spam.

From the simplest filter to a language model



- Each rung fixes a **weakness** of the previous one.
Today: rungs 1 to 5 (notebooks notebooks/01-spam/00 to 04).
- We keep the **same data, split and metrics** on every rung, so the numbers are comparable.

Tokenisation and normalisation

URGENT! You won a £1000 prize. Call
09061701461 now !!

↓ lower-case, digit runs → num, keep ! and £

urgent ! you won a £ prize call num now ! !

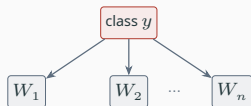
- **Bag of words:** keep *which* words and *how often*, forget the order.
- **Design choices matter:** £, ! and phone-number *shape* are strong spam signals; a naive `isalpha()` filter throws them away.
- Vocabulary from the **training set only** (words in ≥ 2 messages: 3,022).

Bayes' theorem for text

A message is a list of words $W_1, \dots, W_n$; the class is $y \in \{\text{spam}, \text{ham}\}$.

$$P(y \mid W_1..W_n) = \frac{P(y) P(W_1..W_n \mid y)}{P(W_1..W_n)}$$

Naive assumption: words are independent *given the class*, so $P(W_1..W_n \mid y) = \prod_i P(W_i \mid y)$.



Rung 1: discrete Naive Bayes by hand

Presence of a word, Laplace smoothing $k = 1$:

$$P(w \mid y) = \frac{\#\{y\text{-messages containing } w\} + 1}{\#\{y\text{-messages}\} + 2}$$

Toy set: 4 spam, 3 ham; $P(\text{spam}) = 0.571$.

word	$P(w \mid \text{spam})$	$P(w \mid \text{ham})$
password	0.500	0.200
your	0.667	0.400
activity	0.167	0.600
renew (unseen)	0.167	0.200

- send us your password $\rightarrow 0.979$; renew your vows $\rightarrow 0.436$: **wrong** (no meaning).

Rung 2: hard frequencies (counts)

Multinomial model: a message is a **bag of n tokens**; use word *counts* x_w .

$$P(w \mid y) = \frac{N_{w,y} + k}{\sum_{w'} N_{w',y} + k |V|}, \quad \log P(y \mid \mathbf{x}) \propto \log P(y)$$

- The score is a **dot product** of the count vector with a table of log-probabilities.
- On the real corpus (notebooks/01-spam/01):

Model	Precision	Recall	F1	MCC
NB, presence ($k = 1$)	0.953	0.946	0.949	0.942
NB, counts ($k = 1$)	0.961	0.953	0.957	0.951

Logarithms: numerical stability

A 1,007-token message: the product of $\sim 1,000$ probabilities underflows.

Method	$P(\text{spam})$ and $P(\text{ham})$ (unnormalised)	Posterior
Product of probabilities	0.0 and 0.0	$0/0 = \mathbf{NaN}$
Sum of log-probabilities	-6114.6 and -7651.0	≈ 1 (spam)

- $\log ab = \log a + \log b$: **multiplications become sums.**
- Normalise with **log-sum-exp**:
 $\log \sum_y e^{\ell_y} = m + \log \sum_y e^{\ell_y - m}, m = \max_y \ell_y.$
- Never exp a very negative number alone;
`scipy.special.logsumexp` does the shift.

Evaluate like a defender

	predicted ham	predicted spam
actual ham	TN	FP: real message blocked
actual spam	FN: spam in the inbox	TP: spam caught

$$\text{precision} = \frac{TP}{TP + FP} \quad \text{recall} = \frac{TP}{TP + FN} \quad F_1 = \frac{2PR}{P + R}$$

- Users forgive some spam, not a lost message: **push precision up**, read the recall. Naive Bayes at threshold 0.9914: precision **0.992**, recall **0.930**.

Term frequency and inverse document frequency

Counts have two problems: **frequent words dominate** (you, to, the), and **long messages dominate**.

$$\text{tf-idf}(t, d) = (1 + \ln c_{t,d}) \cdot \left(\ln \frac{1+N}{1+\text{df}(t)} + 1 \right), \quad \mathbf{x}_d \leftarrow \frac{\mathbf{x}_d}{\|\mathbf{x}_d\|_2}$$

- **Sub-linear tf** (log): the 10th repetition matters less than the 2nd.
- **idf**: weight by rarity (Spärck Jones 1972). The lowest-idf words: i, to, you, a, the, !, u.
- **L2 normalisation**: every message has length 1, so length no longer matters.

Does TF-IDF help Naive Bayes?

Model	Precision	Recall	F1	PR-AUC
Multinomial NB, counts	0.953	0.953	0.953	0.974
Multinomial NB, TF-IDF	0.968	0.930	0.949	0.983
Complement NB, TF-IDF	0.873	0.961	0.915	0.983

- **Honest result:** F1 is the same within the noise; the *ranking* (PR-AUC) improves.
- χ^2 **vocabulary selection** keeps the informative words: num, p, txt, claim, free, call, mobile, prize, www, stop.
- Those are also the words an attacker must **disguise**.

From generative to discriminative

Naive Bayes models *how each class generates words*. Logistic regression learns **one weight per word** directly:

$$z = \mathbf{w} \cdot \mathbf{x} + b, \quad P(\text{spam} \mid \mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

- Both give a **linear** score: NB's weights are log-odds $\log \frac{P(w|\text{spam})}{P(w|\text{ham})}$, LR's are optimised.
- Loss (cross-entropy) with L2: $J = -\frac{1}{n} \sum_i [y_i \log \sigma(z_i) + (1 - y_i) \log \sigma(-z_i)] + \frac{\lambda}{2} \|\mathbf{w}\|^2$.

Regularisation and training

- The L2 term $\frac{\lambda}{2} \|\mathbf{w}\|^2$ is **regularisation**: it shrinks the weights, so rare words cannot be memorised (*overfitting*). scikit-learn's C is $1/\lambda$: small C, strong regularisation; we choose it by cross-validation on the training set.
- **Stable** form: $\log_sigmoid$; gradient $\frac{1}{n} X^\top (\sigma(X\mathbf{w} + b) - \mathbf{y}) + \lambda\mathbf{w}$, trained with **gradient descent** or **L-BFGS**.

What the weights say

Pushes to spam	weight	Pushes to ham	weight
num (phone number)	20.8	i	-6.0
p (pence)	8.0	my	-4.0
text, txt	7.7, 7.6	me	-3.1
mobile	6.8	gt, lt	-3.0
won, free	5.9, 5.7	can, he	-2.7, -2.5

- gt and lt are HTML escapes (< ; #> ;) in the *ham* half of the corpus: a **spurious correlation**, an artefact of data collection (Arp et al. 2022).
- The weights are also the **attacker's roadmap**.

Which model family?

5-fold cross-validation on the training set, TF-IDF *inside* each fold (mean $\pm$ std of F1):

Model	words 1-2 grams	characters 2-5 grams
Multinomial NB	0.947 $\pm$ 0.012	0.946 $\pm$ 0.009
Logistic regression	0.955 $\pm$ 0.015	0.955 $\pm$ 0.009
Linear SVM	0.956 $\pm$ 0.009	0.962 $\pm$ 0.010
Random forest	0.933 $\pm$ 0.016	0.940 $\pm$ 0.016
k-NN (cosine)	0.917 $\pm$ 0.027	0.908 $\pm$ 0.015
MLP (128)	0.914 $\pm$ 0.034	0.932 $\pm$ 0.017

- Differences below the $\pm$ are **noise**. Linear models on sparse TF-IDF are hard to beat.

Four rungs, one table

Rung	Model	Precision	Recall	F1
1	NB presence	0.953	0.946	0.949
2	NB counts	0.961	0.953	0.957
3	NB TF-IDF	0.968	0.930	0.949
4	Logistic regression (TF-IDF)	0.975	0.907	0.940

- All within **0.94 to 0.96** on this easy corpus: the clean test score hardly separates them.
- What differs is what they *know*: every model above sees a message as **a bag of exact words**.

What a bag of words cannot do

- **No similarity:** cash, money and prize are as unrelated as cash and banana.
- **No out-of-vocabulary handling:** fr33, fre3 and freee are unknown words, their evidence is lost.
- **Next class:** word vectors (Word2Vec, fastText), then sentence embeddings from small language models served by llama.cpp.
- **The class after:** the filters under attack: which patterns bypass them, and which of those still work for a scammer.

Labs and practice guides

Notebook
(notebooks/01-spam/)

Rung

00, 01

discrete and count-based Naive Bayes, log-space, evaluation

02, 03

TF-IDF, vocabulary selection, logistic regression from scratch (JAX)

04

fair comparison of the classical models

- **Practice guide** (practice/spam/guide_02_01): Naive Bayes, TF-IDF and logistic regression with JAX, in a notebook to fill in. Next: guide_02_02 (word vectors and SLM embeddings), guide_02_03 (attacks).

Take-aways

- Spam detection is a **classification problem with an adversary**: the data moves when the filter changes.
- The ladder so far: **counts** → **logs** → **TF-IDF** → **linear**. Each step fixes a weakness; on clean data the scores are close.
- What separates the rungs is **cost, label efficiency and robustness**, not the headline F1.
- Every detector is an **optimisation target** for the attacker: model the threat, then measure the attack (class 4).
- **Next class**: from words to meaning: Word2Vec, fastText and small-language-model embeddings.

Bayes and the evidence by total probability, then conditional independence of the words given the class:

$$P(y \mid \mathbf{w}) = \frac{P(y) \prod_i P(w_i \mid y)}{\sum_{y'} P(y') \prod_i P(w_i \mid y')}$$

- **Bernoulli / presence** model: a word counts once per message (df counts).

- **Multinomial** model: a message is n draws from a class-specific word distribution; the likelihood uses the counts x_w :

$$P(\mathbf{x} \mid y) \propto \prod_w P(w \mid y)^{x_w} \quad \Rightarrow \quad \ell_y = \log P(y) + \sum_w x_w \log$$

- The **decision** for two classes depends only on the sign of $\ell_{\text{spam}} - \ell_{\text{ham}}$.

Laplace smoothing is a Bayesian prior

Maximum likelihood gives $\hat{p}_w = N_{w,y}/N_y$: a word never seen in spam has probability 0 and **veto**es the whole message.

With a symmetric Dirichlet prior (Beta for one word) with parameter k , the **posterior mean** is

$$\hat{p}_w = \frac{N_{w,y} + k}{N_y + k|V|}$$

- $k = 1$ is Laplace; $k < 1$ (Lidstone) trusts the data more. Small $k \Rightarrow$ more extreme posteriors.

- k is a **hyper-parameter**: choose it by cross-validation on the training set, never on the test set.

Softmax over two scores:

$$P(y \mid \mathbf{x}) = \frac{e^{\ell_y}}{e^{\ell_0} + e^{\ell_1}} = \sigma(\ell_1 - \ell_0).$$

Shifting by $m = \max_y \ell_y$ changes nothing but keeps every exponent ≤ 0 :

$$\log \sum_y e^{\ell_y} = m + \log \sum_y e^{\ell_y - m}$$

- Naive version: e^{-6114} and e^{-7651} are both 0 in `float64` (smallest positive $\approx 10^{-308}$), so 0/0.

- Stable version:
 $\sigma(-6114.6 + 7651.0) = \sigma(1536) = 1.$
- The same trick stabilises the **log-sigmoid** and **cross-entropy** losses.

Naive Bayes is a linear classifier

For two classes, the log-odds is **linear** in the counts:

$$\log \frac{P(\text{spam} \mid \mathbf{x})}{P(\text{ham} \mid \mathbf{x})} = \underbrace{\log \frac{P(\text{spam})}{P(\text{ham})}}_b + \sum_w x_w \underbrace{\log \frac{P(w \mid \text{spam})}{P(w \mid \text{ham})}}_{w_w}$$

- Same form as logistic regression, $z = \mathbf{w} \cdot \mathbf{x} + b$; only the **way the weights are chosen** differs.
- Naive Bayes weights come from counting (generative, one pass); logistic-regression weights minimise the classification loss (discriminative).

- **Correlated words** are counted twice by NB:
over-confident posteriors, but often good rankings.

Why the weights look like this

- **idf as information:** a word in a fraction $p = \text{df}/N$ of the documents carries $-\ln p = \ln \frac{N}{\text{df}}$ nats when it occurs (Spärck Jones 1972).
- **Sublinear tf** $1 + \ln c$: repeated words give diminishing returns (a concave function of the count).
- **L2 normalisation:** $\|\mathbf{x}\|_2 = 1$ makes the dot product a **cosine similarity**, independent of message length.

- For Multinomial NB, TF-IDF acts as **fractional counts**; Complement NB and other corrections address skewed classes (Rennie et al. 2003).

Variant	Formula
raw tf	$c_{t,d}$
relative tf	$c_{t,d} / \sum_{t'} c_{t',d}$
sublinear tf	$1 + \ln c_{t,d}$
smooth idf	$\ln \frac{1+N}{1+df} + 1$

Training as maximum a posteriori estimation

Bernoulli likelihood of the labels:

$\prod_i \sigma(z_i)^{y_i} (1 - \sigma(z_i))^{1-y_i}$. Its negative log is the **cross-entropy**.

- An **L2 penalty** $\frac{\lambda}{2} \|\mathbf{w}\|^2$ is a Gaussian prior on $\mathbf{w}$ (MAP estimate); **L1** is a Laplace prior and gives **sparse** weights.
- The loss is **convex**: gradient descent and L-BFGS reach the same global minimum.
- Gradient: $\nabla_{\mathbf{w}} J = \frac{1}{n} X^\top (\sigma(X\mathbf{w} + b) - \mathbf{y}) + \lambda \mathbf{w}$, using $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.
- Stable loss: $\log \sigma(z) = -\text{softplus}(-z)$ and $\log(1 - \sigma(z)) = \log \sigma(-z)$.

Optimisers on the same loss

	Gradient descent	L-BFGS
Update	$\theta \leftarrow \theta - \eta \nabla J$	quasi-Newton, curvature from past gradients
Needs	gradient, a learning rate	gradient only, no learning rate
Iterations	300 in our lab	about a tenth
Memory	$O(d)$	$O(md)$, small m

- Learning rate on the L2-normalised TF-IDF features: $\eta = 50$; the features are tiny, so the gradients are too.
- Convergence check: the analytic gradient matches JAX autodiff to 10^{-17} .

Precision at a prevalence

TPR = recall, $FPR = FP / (FP + TN)$. Precision depends on the **prevalence** π :

$$\text{precision} = \frac{\text{TPR} \cdot \pi}{\text{TPR} \cdot \pi + \text{FPR} \cdot (1 - \pi)}$$

- On our test set $\pi = 12.6\%$; in a real mailbox spam can be far more (or less) frequent: **re-compute precision at the deployment prevalence.**
- ROC curves do not depend on π and can look excellent when precision is poor; **PR curves** do depend on it.
- **MCC** uses all four cells and is informative even for very imbalanced data.

Comparing models honestly

- A single test score is a **random variable**: report the mean $\pm$ standard deviation over folds (5-fold CV) or seeds.
- In our comparison the folds spread by ± 0.01 F1: differences of a few thousandths are **noise**.

- **McNemar's test** compares two models on the *same* test set from the discordant pairs b (A right, B wrong) and c (A wrong, B right):

$$\chi^2 = \frac{(|b - c| - 1)^2}{b + c} \quad (1 \text{ degree of freedom})$$

- Fit vectorisers, IDF, scalers and embeddings on the **training folds only** (put them in a pipeline).

Summary

- Naive Bayes and logistic regression are both **linear scores**; they differ in how the weights are fitted.
- **Numerical stability** (log space, log-sum-exp, log-sigmoid) is part of the algorithm, not a detail.
- Report **precision at the deployment prevalence** and the **spread** over folds; use paired tests to compare.
- Bags of words cannot express similarity or handle unseen spellings: the motivation for word vectors and embeddings (next class).
- An attacker reads the same weights: **linear scores are transparent** to the white-box attacker (class 4).

Almeida, Tiago A., José María Gómez Hidalgo, and Akebo Yamakami. 2011. "Contributions to the Study of SMS Spam Filtering: New Collection and Results." *ACM Symposium on Document Engineering (DocEng)*.

Arp, Daniel, Erwin Quiring, Feargus Pendlebury, et al. 2022. "Dos and Don'ts of Machine Learning in Computer Security." *31st USENIX Security Symposium*.

Graham, Paul. 2002. *A Plan for Spam*.
<https://paulgraham.com/spam.html>.

Joulin, Armand, Edouard Grave, Piotr Bojanowski, and Tomas Mikolov. 2017. "Bag of Tricks for Efficient Text Classification." *Conference of the European Chapter of the Association for Computational Linguistics (EACL)*.

Reimers, Nils, and Iryna Gurevych. 2019. "Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks." *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Rennie, Jason D. M., Lawrence Shih, Jaime Teevan, and David R. Karger. 2003. "Tackling the Poor Assumptions of Naive Bayes Text Classifiers." *International Conference on Machine Learning (ICML)*.

Spärck Jones, Karen. 1972. "A Statistical Interpretation of Term Specificity and Its Application in Retrieval." *Journal of Documentation* 28 (1): 11–21.